

Part 13: How to Think About Tree Problems

Recognising the pattern before writing code

This is the most important part of the course. Everything before it was equipment. This is how you choose which piece of equipment to pick up.

The ten questions

When you open a tree problem you have never seen, go through this list. It takes about a minute and it decides the entire solution.

- 1. Do I need DFS or BFS?** Does the answer care about levels, distance from the root, or “nearest”? BFS. Does it care about subtree structure? DFS.
- 2. Does the answer at a node depend on its left and right subtrees?** If yes, you are writing `combine(L, R, node)` and the recursion is postorder.
- 3. Should information travel upward or downward?** Upward: return a value (height, count, sum, best-so-far). Downward: add a parameter (depth, running sum, allowed range, path so far). Sometimes both.
- 4. Is this a level based problem?** Any mention of level, row, “each level”, left or right view, zigzag, or level sums means BFS with the size snapshot.
- 5. Am I being asked to avoid recursion?** Then simulate it with an explicit stack. Preorder and inorder templates from Part 4.
- 6. Do I need a horizontal coordinate?** Top, bottom, vertical, column, “looking from above”. Carry `hd` alongside the node, group with a `map`.
- 7. Is this a path problem, and can the path turn a corner?** Root to leaf, no turn: carry state down. Any node to any node, turns once: bottom-up with the record/return split.
- 8. Is this a subtree problem?** “Contains”, “identical”, “largest subtree such that ...”. Usually a helper comparing two nodes at once, or a bottom-up function returning a bundle of facts.
- 9. Can I decide everything at a single node using only $O(1)$ information from each child?** If yes, one pass is enough and the answer is $O(N)$. If not, decide what larger bundle each child should return.
- 10. Am I repeating work?** Calling a helper that walks a subtree, from inside a recursion that already walks that subtree, is the $O(N^2)$ signature. Merge the two passes.

The decision guide

Does the problem mention levels, rows, views by level, or "nearest"?

yes -> BFS with the size snapshot template
no -> continue

Does it mention columns, vertical, top view, bottom view?

yes -> BFS carrying horizontal distance + map
no -> continue

Is it a path that may turn at some node (max path sum, diameter)?

yes -> bottom-up: record "through me", return "one side only"
no -> continue

Is it a root-to-leaf path, or does each node need context from above?

yes -> preorder with an extra parameter, backtrack if you keep a shared path
no -> continue

Does my answer need facts about both subtrees?

yes -> postorder: L = solve(left), R = solve(right), combine
no -> plain traversal, do the work at each node

Roughly ninety percent of Easy and Medium tree problems land in one of those five buckets.

Pattern to problem map

Pattern	Signature in the statement	Typical problems
Simple DFS	count, sum, search, modify every node	Count Nodes, Invert Tree, Merge Trees
Traversal	"return the preorder / inorder"	144, 94, 145
BFS by level	level, row, view, zigzag, average	102, 107, 103, 199, 637, 515
Bottom-up DP	height, depth, balanced, diameter, "longest / best path"	104, 110, 543, 124, 687
Path recursion	root to leaf, path sum, all paths	112, 113, 129, 257
Two-tree recursion	same, mirror, merge, contains	100, 101, 617, 572
Ancestor problems	lowest common ancestor, distance	236, 235, 1123
Horizontal distance	top view, bottom view, vertical	987, 314, GFG views

Construction	build from traversals, serialize	105, 106, 297, 108
BST specific	sorted, kth smallest, validate, search	98, 230, 700, 701, 235

Applying the checklist to three unseen problems

Example 1: “Return the sum of values of all nodes with a grandparent whose value is even” (LeetCode 1315)

- Levels? No.
- Depends on subtrees? Not really, the answer is a plain sum.
- Information direction? **Downward**. Each node needs to know about its parent and grandparent.
- So: preorder with two extra parameters.

```
int dfs(TreeNode* root, int parent, int grandparent) {
    if (root == NULL) return 0;
    int add = (grandparent % 2 == 0) ? root->val : 0;
    return add + dfs(root->left, root->val, parent)
        + dfs(root->right, root->val, parent);
}
// call dfs(root, 1, 1) using odd sentinels so the top two levels contribute nothing
```

The whole problem was solved by question 3.

Example 2: “Find the largest subtree that is a BST”

- Levels? No.
- Depends on subtrees? Yes, strongly.
- Can each child give me O(1) information? Let us see. To know whether I am a BST I need: is my left subtree a BST, is my right subtree a BST, is the **maximum** of my left subtree less than me, is the **minimum** of my right subtree greater than me. And to report a size I need their sizes.
- So each node returns four things: {isBST, minVal, maxVal, size}.

That is question 9 doing its job. The moment you list what you need, the return type designs itself, and the code is then mechanical.

Example 3: “Print all nodes at distance K from a target node”

- Levels? Yes, but distance from an arbitrary node, not from the root.
- The obstacle: distance goes both down and **up**, and a tree has no parent pointers.
- Fix: one preorder pass to record each node’s parent in a map, then BFS outward from the target treating the tree as an undirected graph, with a visited set.

Question 3 exposed the obstacle (information must move upward through edges that only point down), and the fix follows immediately. This is LeetCode 863, a Medium that many people find hard purely because they never asked which direction the information travels.

When you are stuck

In order, these are the moves that work:

1. **Draw a tree with 5 to 7 nodes and solve it by hand.** Not in your head. On paper. Then ask what you actually did.
2. **Write down what you wish each subtree would tell you.** Then that is the return type.
3. **Answer the base case honestly.** What is this quantity for an empty tree?
4. **Solve the two node case, then the three node case.** Bugs show up immediately.
5. **Say the function's meaning out loud in one sentence.** If you cannot, the code is guesswork. “f(x) returns the height of x's subtree, or -1 if it is unbalanced” is a sentence. “f does the balance thing” is not.
6. **Check for repeated work.** If a helper walks the subtree inside a recursion that walks the subtree, merge them.

Habits that separate strong students

- They write the base case first, always.
- They name the return value before writing the body.
- They keep “the answer through this node” and “the value returned to my parent” as two separate variables, even when they happen to be equal.
- They test on a skewed tree, a single node, and an empty tree, not just the pretty balanced example in the problem statement.
- They reset globals at the start of the public function.
- They state the complexity out loud before submitting, and check that it matches the constraints.

Check yourself

1. A problem asks for the average value of the nodes on each level. Which bucket is it, and what template do you reach for?
2. A problem asks for the length of the longest path where all node values are equal. Which bucket, and what does each node return upward?
3. A problem asks whether a tree contains a root to leaf path whose values spell a given word. Which direction does information flow?
4. A problem asks for the node values visible when looking at the tree from directly above. What coordinate do you need, and what traversal?
5. You have written a solution that calls `height()` at every node. What should you check before submitting?
6. Why is “state the meaning of the return value in one sentence” such a useful discipline?